# Lecture 16

2025-12-17

Feature-based models: self-organizing map, elastic net; Supervised learning: binary classification and function approximation

- Online slides: lec16-feature_based.html
- Code: code16.ipynb, lec16-data.npz
- Homework: hw16.pdf

Username: cns, Password: nycu2025

Laboratory of Computing Biological Networks

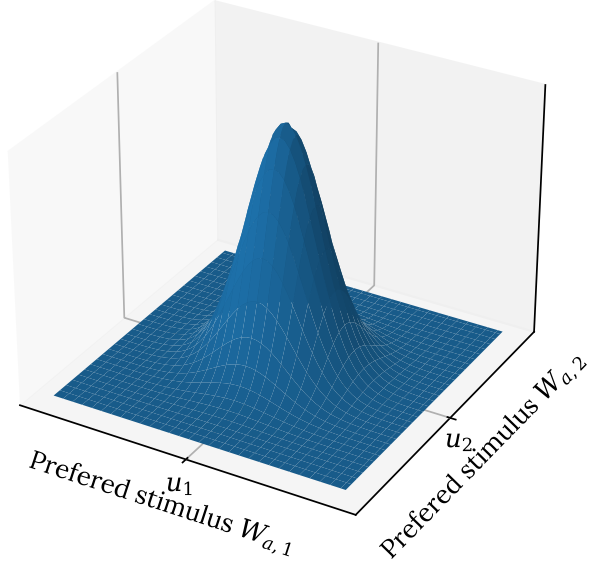
# Feature-based model

*– Departure from firing rates and synaptic strengths*

Instead of weight, we consider $W_{ab}$ as selectivity of output unit $a$ to stimulus feature $b$. For example, retinotopic input features can be $\mathbf{u} = (x, y, o, e\cos\theta, e\sin\theta)$.

## Competition activity variable

$$z_a = \frac{\exp\left(-\sum_b \left(u_b - W_{ab}\right)^2 / \left(2\sigma_b^2\right)\right)}{\sum_{a'} \exp\left(-\sum_b \left(u_b - W_{a'b}\right)^2 / \left(2\sigma_b^2\right)\right)}$$




- Gaussian tuning curves with maximum given by $W_{ab}$ and standard deviation given by $\sigma_b$.
- Total activity is normalized to 1.

# Plasticity rules for competitive FBM

## Self-organizing map

$$v_a = \sum_{a'} M_{aa'} z_{a'}$$

$$\tau_w \frac{dW_{ab}}{dt} = \langle v_a (u_b - W_{ab}) \rangle$$

**M**: usually purely excitatory and fairly short range.

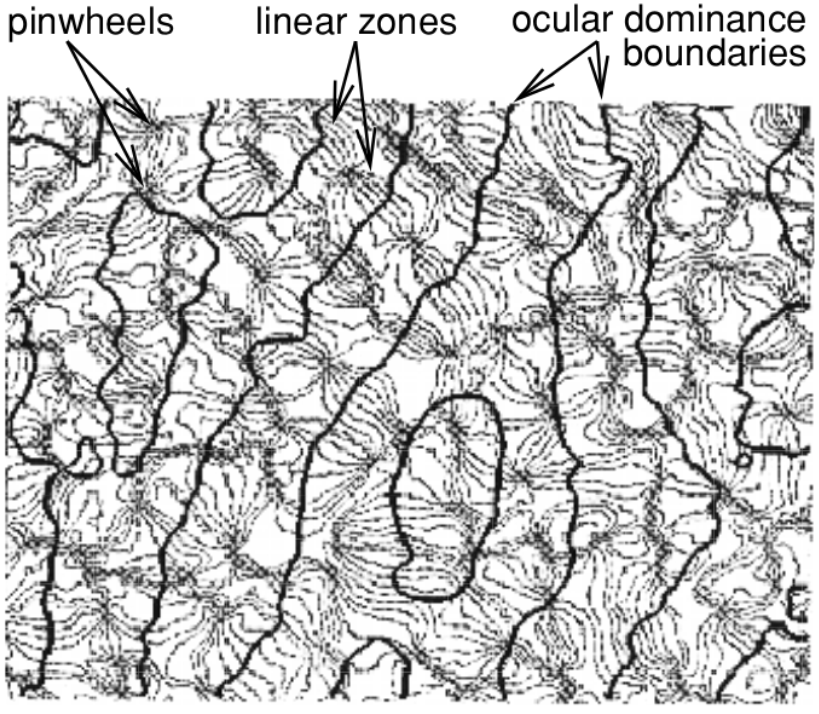
## Elastic net

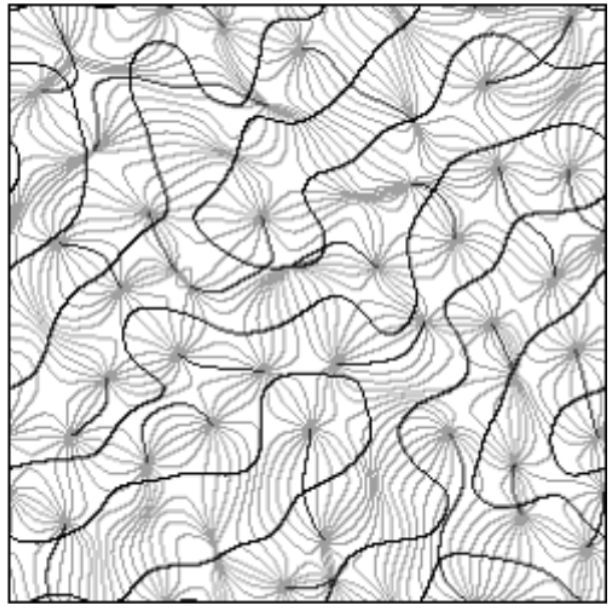
$$v_a = z_a$$

$$\tau_w \frac{dW_{ab}}{dt} = \langle v_a (u_b - W_{ab}) \rangle + \beta \sum_{a' \in \mathrm{nb}(a)} (W_{a'b} - W_{ab})$$

$\mathrm{nb}(a)$: Set of neighbors of $a$.

# Orientation domains and ocular dominance



Left: 1.7×1.7mm patch of macaque V1 (Obermayer & Blasdel 1993)

Right: Elastic net results (Erwin et al. 1995)

Black lines: boundaries of ocular dominance

Gray lines: iso-orientation contours at intervals of 11.25°

# Anti-Hebbian modification

*For dealing with* redundancy *among output neurons*

Oja rule: $\tau_w \frac{dW_{ab}}{dt} = v_a u_b - \alpha v_a^2 W_{ab}$ ⇒ Identical outputs...

Preventing same eigenvector for all outputs ⇐ plastic recurrent connections that is anti-Hebbian

$$\tau_M \frac{d\mathbf{M}}{dt} = -\mathbf{v}\mathbf{v} + \beta\mathbf{M} \quad \text{or} \quad \tau_M \frac{dM_{aa'}}{dt} = -v_a v_{a'} + \beta M_{aa'}$$

Preventing vanishing weights...

Ideally with suitable $\beta$ and $\tau_M$, rows of $\mathbf{W}$ will be different eigenvectors of $\mathbf{Q}$ while $\mathbf{M}$ will ultimately vanish.

# Goodall rule

$$\tau_M \frac{d\mathbf{M}}{dt} = -(\mathbf{W} \cdot \mathbf{u})\mathbf{v} + \mathbf{I} - \mathbf{M}$$

Recurrent network: $\mathbf{v} = \mathbf{W} \cdot \mathbf{u} + \mathbf{M} \cdot \mathbf{v} \Rightarrow \mathbf{v} = \mathbf{K} \cdot \mathbf{W} \cdot \mathbf{u}$ where $\mathbf{K} = (\mathbf{I} - \mathbf{M})^{-1}$.

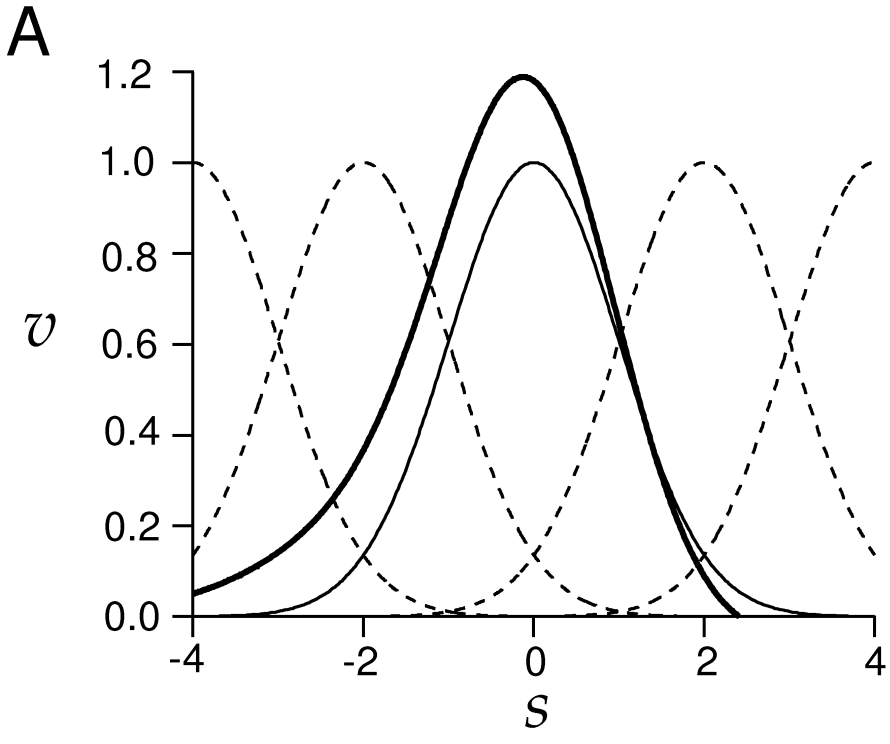
Stationary state:

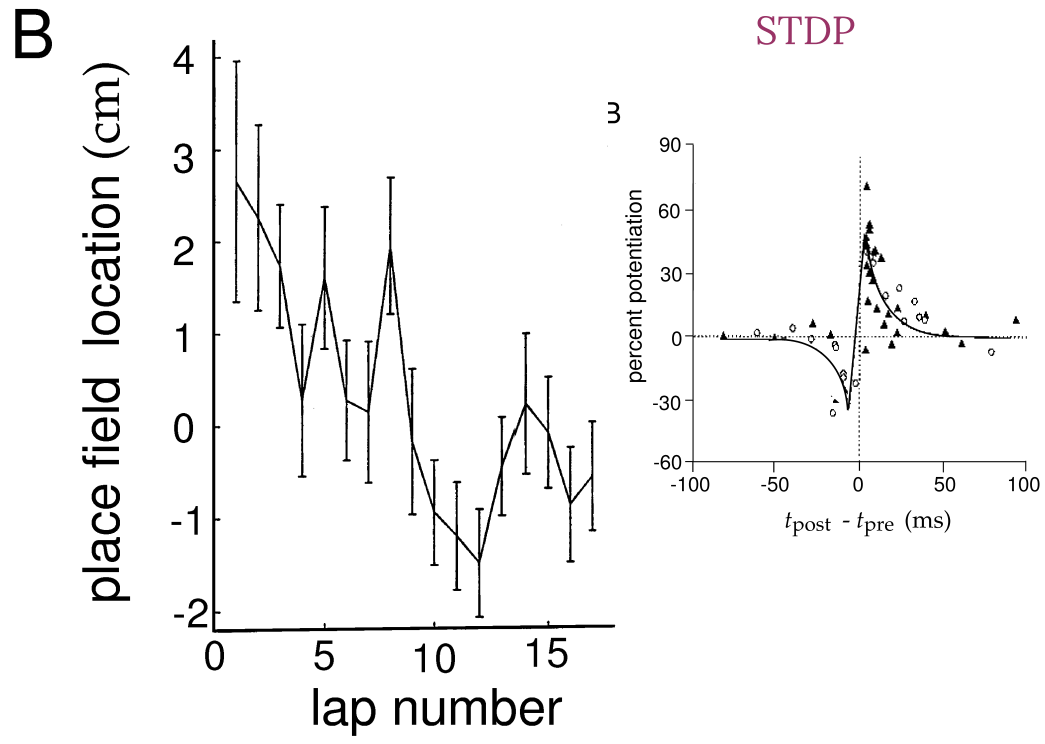
$$\langle (\mathbf{W} \cdot \mathbf{u})\mathbf{v} \rangle = \mathbf{I} - \mathbf{M} = \mathbf{K}^{-1} \Rightarrow \langle \mathbf{v}\mathbf{v} \rangle = \langle (\mathbf{K} \cdot \mathbf{W} \cdot \mathbf{u})\mathbf{v} \rangle = \mathbf{I}$$

- Outputs are decorrelated
- Histogram is equalized
- Linear ⇒ removing upto second-order redundancy

# Timing-based plasticity and prediction

## — *Shifts of place fields*




- Stimulus with a positive rate of change (moving rightward as a function of time)
- A tuning curve asymmetrically broadens and shifts from temporally asymmetric Hebbian plasticity
- Backward along the track relative to direction of motion

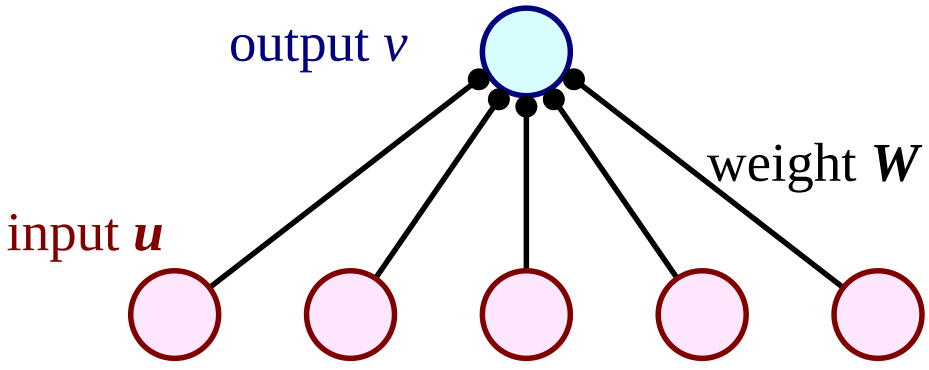
# Learning in neural networks

- Unsupervised learning
  Only input data is given

- Supervised learning
  Both input and expected output data are given
  **Keys**: storage of relationships, generalization to unseen inputs
  Will consider: classification (discrete) & function approximation (continuous, aka regression)

- Reinforcement learning
  Given an environmental condition, an action is chosen and performed by system, the environment changes as a result, based on which, rewards will be claimed occasionally

# Supervised Hebbian learning




Training samples:

$\mathbf{u}^m, v^m$ for $m = 1 \ldots N_s$

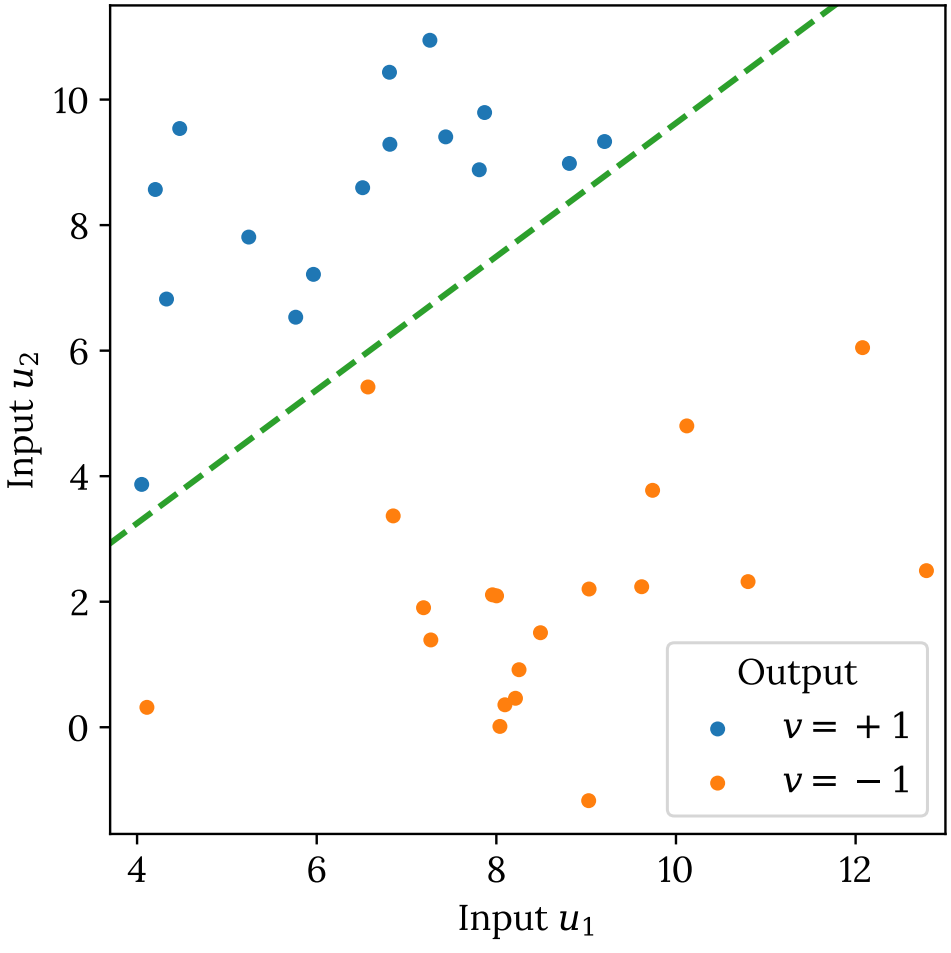
$$\tau_w \frac{d\mathbf{w}}{dt} = \langle v\mathbf{u} \rangle = \frac{1}{N_S} \sum_{m=1}^{N_S} v^m \mathbf{u}^m$$

Imposed externally

With stabilization:

$$\tau_w \frac{d\mathbf{w}}{dt} = \langle v\mathbf{u} \rangle - \alpha \mathbf{w} \quad \Rightarrow \quad \mathbf{w} = \frac{\langle v\mathbf{u} \rangle}{\alpha}$$

# Classification problem




## Linear separability

Find a hyper plane to separate the data into desired categories.

Not always possible ... Consider random binary components of $\mathbf{u}$ and $v$, for large $N_u$, *"the maximum number of random associations that can be described ... is* $2N_u$*."*

# Perceptron

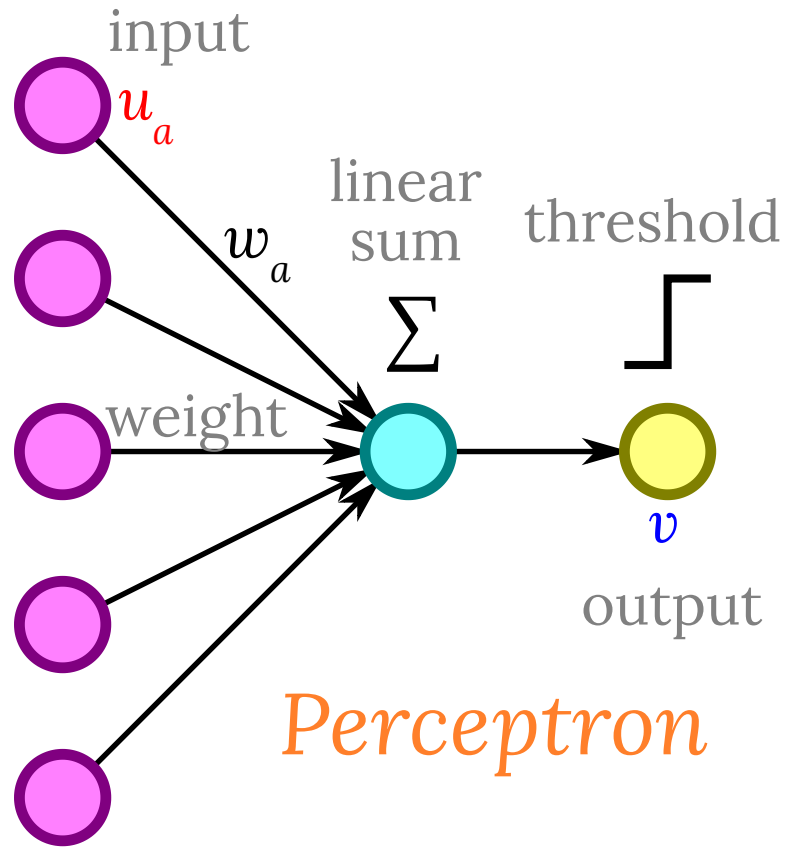
*– a nonlinear map that classifies input into one of two categories*

- Typically consists of a weighted sum over binary inputs that is applied with a threshold to produce a binary output.

- Usually found in the final output layers of artificial neural networks to read out the results.

$$v = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{u} - \gamma \geq 0, \\ -1 & \text{if } \mathbf{w} \cdot \mathbf{u} - \gamma < 0. \end{cases}$$

$\gamma$: the threshold (decision boundary)



# Apply Hebbian learning to perceptron

Hebbian rule with stabilization

$$\tau_w \frac{d\mathbf{w}}{dt} = \langle v\mathbf{u} \rangle - \alpha \mathbf{w} \quad \Rightarrow \quad \mathbf{w} = \frac{\langle v\mathbf{u} \rangle}{\alpha}$$

☆ For linear separable inputs, a set of weights allowing perfect perceptron exists, but may not be achieved by the Hebbian rule...

## An analytical solution

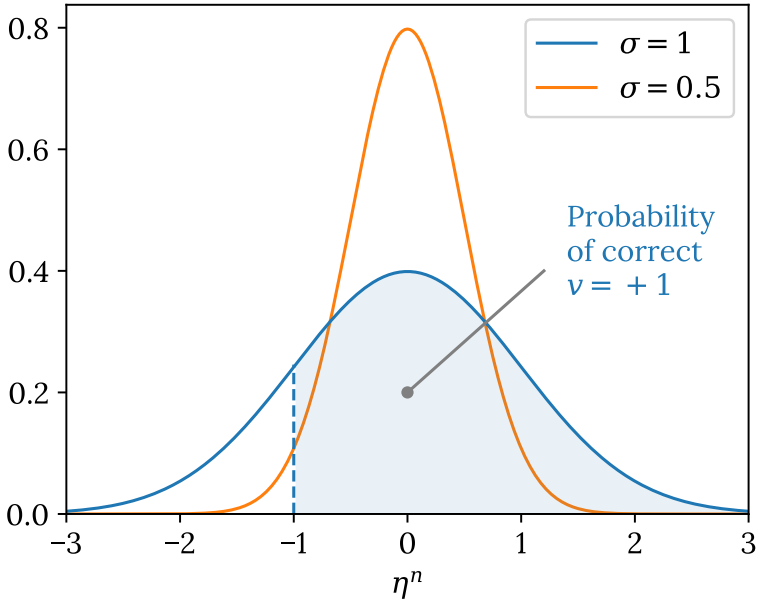
Assuming $\alpha = N_u / N_S$:

$$\mathbf{w} = \frac{1}{N_u} \sum_{m=1}^{N_S} v^m \mathbf{u}^m$$

# Estimate the error from random training data

With $\gamma = 0$, the output of the $n$-th input $\mathbf{u}^n$ is determined by the sign of

$$\mathbf{w} \cdot \mathbf{u}^n = \frac{1}{N_u} \left( v^n \mathbf{u}^n \cdot \mathbf{u}^n + \sum_{m \neq n} v^m \mathbf{u}^m \cdot \mathbf{u}^n \right)$$
$$= v^n + \eta^n$$




$$N_u \eta^n = \sum_{m \neq n} v^m \mathbf{u}^m \cdot \mathbf{u}^n$$

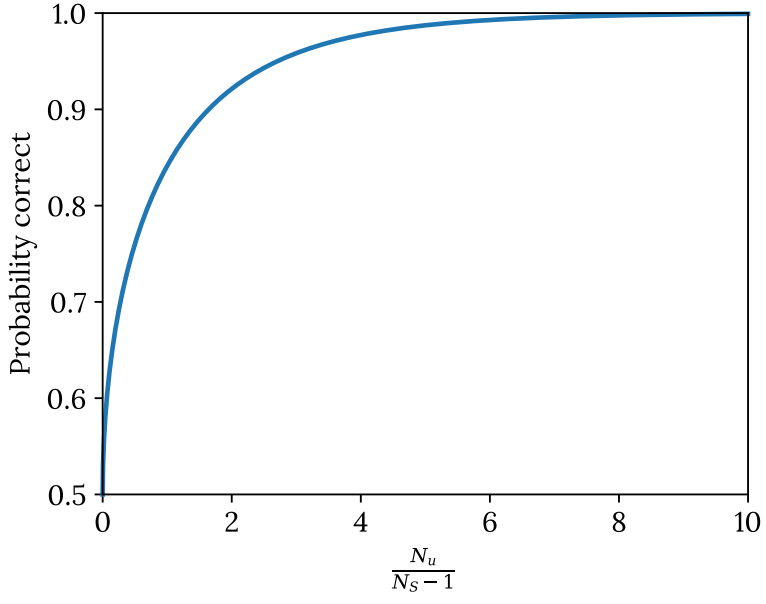
noise-like term from a sum of $(N_S - 1)N_u$ $+1$ or $-1$

$\eta^n$ ~ Gaussian distribution with 0 mean and variance of $(N_S - 1)/N_u$.

# Probability of correct prediction

Combining correct $v^n = +1$ and correct $v^n = -1$:

$$P[\text{correct}] = \sqrt{\frac{N_u}{2\pi(N_S - 1)}} \int_{-\infty}^{1} d\eta \, \exp\left(-\frac{N_u \eta^2}{2(N_S - 1)}\right)$$

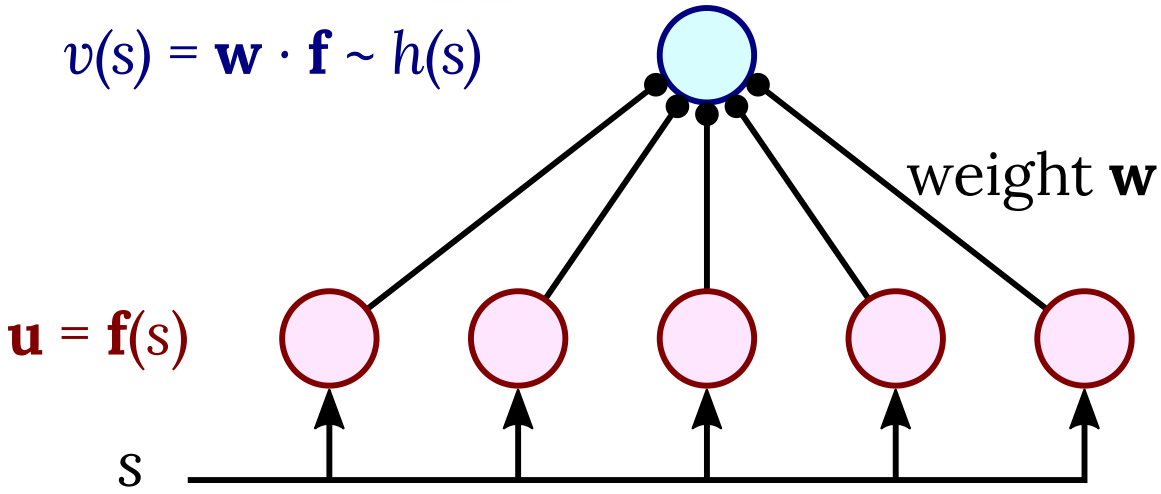



Integral results in a special function:

$$P[\text{correct}] = \frac{1}{2}\left(1 + \text{erf}\left(\sqrt{\frac{N_u}{2(N_S - 1)}}\right)\right)$$

See code16.ipynb for derivation

# Function approximation




Building arbitrary function on $s$ using tuning curves of neurons as basis.

$$v(s) = \mathbf{w} \cdot \mathbf{u} = \mathbf{w} \cdot \mathbf{f}(s) = \sum_{b=1}^{N} w_b f_b(s)$$

$f(s)$ forms a basis for the output function $h(s)$. It's complete if the basis can represent all possible output functions and overcomplete if required weight $w$ is not unique.

# Minimizing the error

$$E = \frac{1}{2}\sum_{m=1}^{N_S}\left(h(s^m) - v(s^m)\right)^2 = \frac{N_S}{2}\left\langle\left(h(s) - \mathbf{w}\cdot\mathbf{f}(s)\right)^2\right\rangle$$

Setting derivative with respect to $w$ zero ⇒

$$\langle \mathbf{f}(s)\mathbf{f}(s)\rangle \cdot \mathbf{w} = \langle \mathbf{f}(s)h(s)\rangle$$

⇒ Normal equations

When will Hebbian solution $\mathbf{w} = \langle \mathbf{f}(s)h(s)\rangle/\alpha$ be correct?

1. Orthogonal condition: $\langle \mathbf{f}(s)\mathbf{f}(s)\rangle = \mathbf{I}$
2. Tight frame condition: $\mathbf{f}(s^m)\cdot\mathbf{f}(s^n) = c\delta_{m,n}$

# Tight frame condition

Substituting the weights $\mathbf{w} = \langle \mathbf{f}(s) h(s) \rangle$ in normal equations

$$\begin{aligned}
\langle \mathbf{f}(s)\mathbf{f}(s) \rangle \cdot \mathbf{w} &= \frac{\langle \mathbf{f}(s)\mathbf{f}(s) \rangle \cdot \langle \mathbf{f}(s) h(s) \rangle}{\alpha} \\
&= \frac{1}{\alpha N_S^2} \sum_{mn} \mathbf{f}(s^m)\mathbf{f}(s^m) \cdot \mathbf{f}(s^n) h(s^n) \\
&= \frac{c}{\alpha N_S^2} \sum_{m} \mathbf{f}(s^m) h(s^m) \\
&= \frac{c}{\alpha N_S} \langle \mathbf{f}(s) h(s) \rangle
\end{aligned}$$

$\Rightarrow \alpha = c/N_S$

# Supervised error-correcting rules

## Perceptron rule

$$\mathbf{w} \to \mathbf{w} + \frac{\epsilon_w}{2} \left(v^m - v(\mathbf{u}^m)\right) \mathbf{u}^m$$

$$\gamma \to \gamma - \frac{\epsilon_w}{2} \left(v^m - v(\mathbf{u}^m)\right)$$

Checking if the rule leads to more correct output...

$$\left(\mathbf{w} \cdot \mathbf{u}^m - \gamma\right) \to \left(\mathbf{w} \cdot \mathbf{u}^m - \gamma\right) + \frac{\epsilon_w}{2} \left(v^m - v\left(\mathbf{u}^m\right)\right) \left(\left|\mathbf{u}^m\right|^2 + 1\right)$$

For fixed $\epsilon_w$, perceptron learning will find a solution to any linearly separable problem.

# Delta rule (gradient descent)

*– for continuous output*

$$\mathbf{w} \to \mathbf{w} - \epsilon_w \nabla_{\mathbf{w}} E \quad \text{or} \quad w_b \to w_b - \epsilon_w \frac{\partial E}{\partial w_b}$$

For function approximation:

$$\nabla_{\mathbf{w}} E = -\sum_{m=1}^{N_S} \left( h(s^m) - v(s^m) \right) \mathbf{f}(s_m)$$

⇒ Update is performed after all samples are shown.

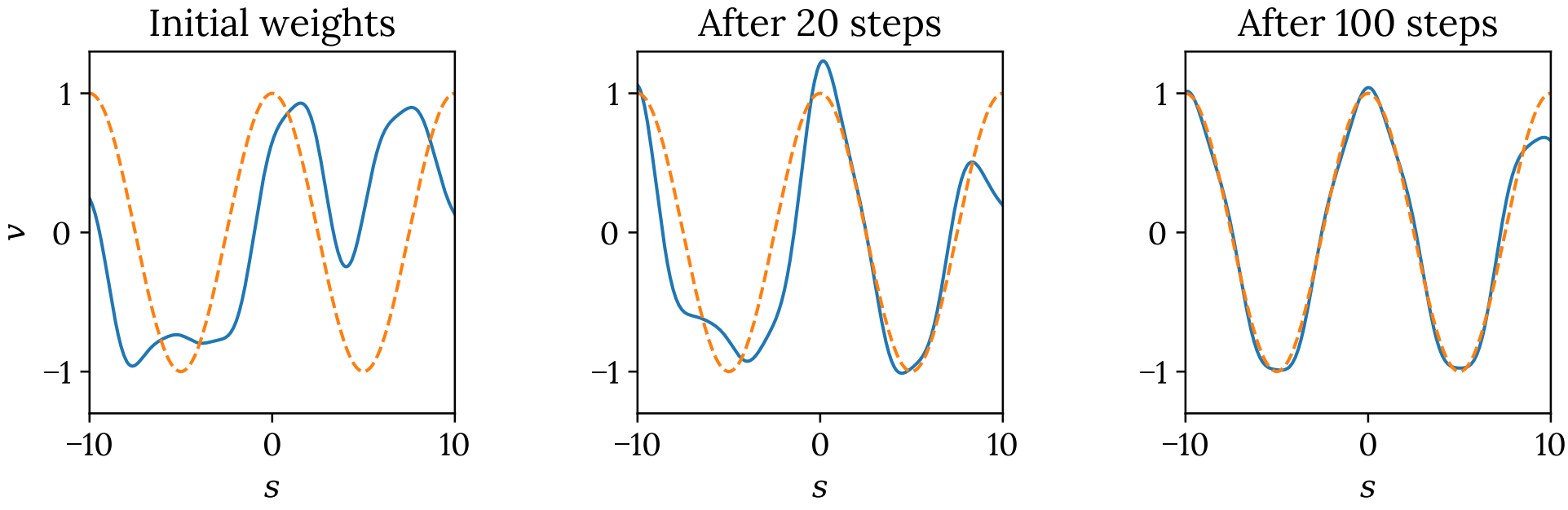
## Stochastic gradient descent (Delta rule)

Update is performed for each random sample:

$$\mathbf{w} \to \mathbf{w} + \epsilon_w \left( h(s^m) - v(s^m) \right) \mathbf{f}(s_m)$$

Full sum replaced by appropriately weighted sum of randomly chosen terms → A form of Monte Carlo method.

# Stochastic gradient descent example




Tuning curves are $f_b(s) = \exp(-(s - s_b)^2/2)$ with $s_b = -10$, $-8$, …, $8$, $10$. Target is a cosine function (dashed).

See the code file code16.ipynb for details.